

Reviewer Pack — Minimal Order of Quasi-Parseval Spaces and Communication Com...

Entry 4da158968540 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 03:33:21 UTC

Minimal Order of Quasi-Parseval Spaces and Communication Complexity Rank Correlation

Entry ID: 4da158968540 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 03:33:21 UTC

1. Conjecture

Field A (mathematical branch): Functional Analysis (Quasi-Parseval Spaces) **Field B** (complexity object): Communication Complexity

Statement:

For every Boolean function $f: \{0,1\}^n \rightarrow \{0,1\}$, the minimal dimension of a quasi-Parseval space associated with f is linearly correlated with its communication complexity rank $r(f)$, such that $\dim(\text{quasi-Parseval_space}(f)) = \Theta(r(f))$.

Rationale (proposer's reasoning):

Quasi-Parseval spaces provide a generalization of Parseval's theorem to non-commutative settings, which may capture the structure of Boolean functions in a way that is relevant to communication complexity. A correlation between these spaces and communication complexity rank could reveal new insights into the nature of communication tasks.

Taxonomy category: functional_analysis_to_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): d2597ee76f07e66b

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if at least 80% of seeds yield a Pearson correlation coefficient between the set of minimal dimensions of quasi-Parseval spaces and their corresponding communication complexity ranks greater than or equal to 0.7, with no seed producing a correlation below 0.5.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Quasi-Parseval Spaces" AND "communication complexity" - "functional analysis" AND "minimal dimension quasi-Parseval spaces" - "Boolean function" AND communication complexity rank AND "quasi-Parseval space dimension"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=240.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def communication_complexity_rank(f):
        n = int(math.log2(len(f)))
        if 2**n != len(f):
            raise ValueError("Invalid function length")

        # Simplified version of communication complexity rank calculation
        return sum(f[i] == f[j] for i in range(n) for j in range(i+1, n)) / (n * (n - 1))

    def quasi_parseval_space_dimension(f):
        n = int(math.log2(len(f)))
        if 2**n != len(f):
            raise ValueError("Invalid function length")

        # Simplified version of quasi-parseval space dimension calculation
        return sum(1 for i in range(n) if f[i] == f[(i + n // 2) % n]) / n

    results = []
    for _ in range(30):
        n = random.choice([5, 10, 15, 20, 30, 40])
        f = generate_boolean_function(n)
        dim_quasi_parseval = quasi_parseval_space_dimension(f)
        r_f = communication_complexity_rank(f)
        results.append((dim_quasi_parseval, r_f))
```

```

if not results:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "empty_results"
    }

dim_quasi_parsevals = [r[0] for r in results]
ranks = [r[1] for r in results]

mean_dim_quasi_parseval = sum(dim_quasi_parsevals) / len(dim_quasi_parsevals)
mean_rank = sum(ranks) / len(ranks)

covariance = sum((dim_quasi_parsevals[i] - mean_dim_quasi_parseval) * (ranks[i] - mean_rank) for i in range(len(results)))
variance_dim_quasi_parseval = sum((dim_quasi_parsevals[i] - mean_dim_quasi_parseval)**2 for i in range(len(results)))
variance_rank = sum((ranks[i] - mean_rank)**2 for i in range(len(results))) / len(results)

if variance_dim_quasi_parseval == 0 or variance_rank == 0:
    return {
        "metric_name": "Pearson correlation coefficient",
        "metric_value": None,
        "instances_tested": len(dim_quasi_parsevals),
        "n_max": max(n for _, n in results),
        "conjecture_holds": False,
        "counterexample": "variance_zero"
    }

pearson_correlation_coefficient = covariance / (math.sqrt(variance_dim_quasi_parseval) * math.sqrt(variance_rank))

return {
    "metric_name": "Pearson correlation coefficient",
    "metric_value": pearson_correlation_coefficient,
    "instances_tested": len(dim_quasi_parsevals),
    "n_max": max(n for _, n in results),
    "conjecture_holds": pearson_correlation_coefficient >= 0.7 and all(r >= 0.5 for r in ranks),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if not results:
        print("RESULT: INCONCLUSIVE no_results")
    else:
        supported_count = sum(1 for r in results if r["conjecture_holds"])
        support_fraction = supported_count / len(results)

```

```

if support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={sum(r['metric_value'] for r in results) / len(results)}")
else:
    first_failing_seed = next(seed for seed, r in enumerate(results, start=min(seeds)) if
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not meet the pre-registered support condition for the conjecture. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	16395
2	propose	ollama_remote	glm4:latest	0	0	14044
3	preregistration	ollama_remote	glm4:latest	0	0	11807
4	novelty	ollama_remote	glm4:latest	0	0	14920
5	novelty	ollama_remote	glm4:latest	0	0	12395
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17799
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16461
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47733
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	25378
10	judge	ollama_remote	glm4:latest	0	0	27415

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 204347 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/4da158968540.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/4da158968540.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/4da158968540.tar.gz` (if generated)